# B.Tech. Cloud / Cloud Native Computing Laboratory

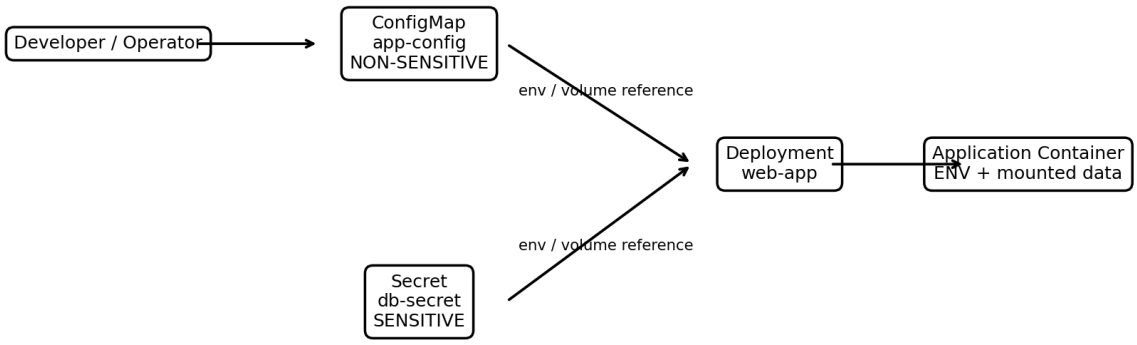
# LABORATORY MANUAL

## Implement Configuration Management in Kubernetes using ConfigMaps and Secrets

Deploy a containerized application that consumes both non-sensitive and sensitive configuration data

| Item | Details |
|---|---|
| Programme | B.Tech. |
| Suggested Duration | 2 Hours |
| Experiment Type | Hands-on / Individual or Pair |
| Platform | Docker Desktop with Kubernetes enabled |
| Alternative | Minikube / local Kubernetes cluster |
| Difficulty | Intermediate |

**Faculty Note:** The lab uses a small Nginx-based container to make the configuration visible. Students should understand that Kubernetes Secrets are intended for sensitive configuration, but storing a value in a Secret does not by itself provide complete secret-management security.




ConfigMap supplies normal settings; Secret supplies sensitive values.
The application consumes both through Kubernetes-managed configuration.

*Figure 1. Application consuming a ConfigMap and a Secret.*

# 1. Experiment Overview

Configuration management separates application code from environment-specific settings and credentials. In Kubernetes, a ConfigMap stores non-sensitive configuration such as application mode, UI title, or log level. A Secret provides a separate object for sensitive values such as passwords, tokens, or keys. A Pod can consume these objects as environment variables or mounted files.

| Configuration Type | Example | Kubernetes Object |
|---|---|---|
| Non-sensitive | APP_MODE=development | ConfigMap |
| Non-sensitive | WELCOME_MESSAGE=Hello from Kubernetes | ConfigMap |
| Sensitive | DB_PASSWORD=<student-created value> | Secret |
| Sensitive | API_TOKEN=<student-created value> | Secret |

# 2. Learning Objectives

- Explain the purpose and difference between ConfigMaps and Secrets.
- Create a ConfigMap and inject configuration into a container as environment variables.
- Create a Secret and consume sensitive data from a container as environment variables.
- Mount configuration and secret data as files inside a container.
- Update configuration objects and verify how Pods respond to configuration changes.
- Use kubectl commands to inspect, validate, and troubleshoot configuration management.
- Apply basic good practices for handling sensitive configuration in Kubernetes.

# 3. Prerequisites

- Basic knowledge of Docker containers, Kubernetes Pods, Deployments, and YAML.
- Docker Desktop installed with Kubernetes enabled, or Minikube/local Kubernetes.
- kubectl available in the terminal.
- Basic YAML knowledge.
- Internet access for the first pull of nginx:alpine.

**Important:** Use a local lab cluster and never place a real production password, API key, or cloud credential in this exercise.

# 4. Concept Review

## 4.1 ConfigMap

A ConfigMap is designed for non-confidential key-value configuration. It can be consumed as environment variables, command-line arguments, or files mounted into a Pod.

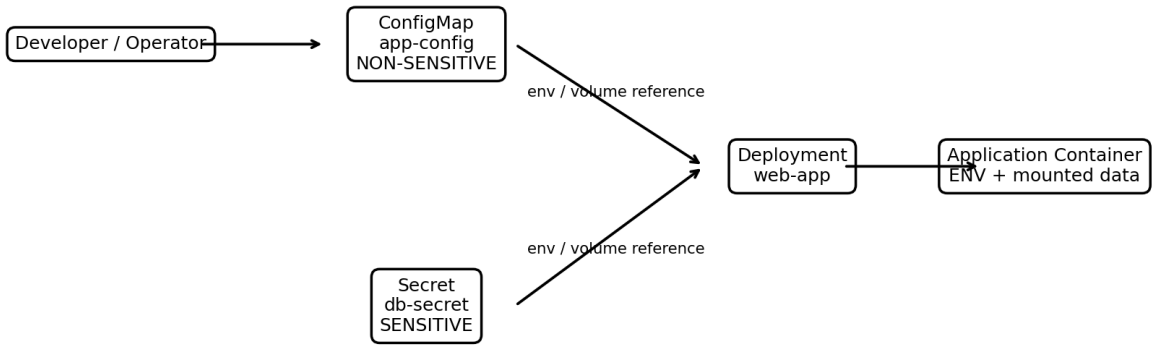
## 4.2 Secret

A Secret is a Kubernetes object for sensitive data. In YAML, values under data are base64-encoded, which is an encoding mechanism rather than encryption. Access to Secrets should therefore be controlled using appropriate Kubernetes permissions and cluster security practices.

## 4.3 Environment Variables vs Mounted Files

| Method | Use in this lab | Advantage | Limitation |
|---|---|---|---|
| Environment variable | ConfigMap + Secret | Easy for applications to read | Application may need restart to consume changed values. |
| Mounted file | ConfigMap + Secret | Useful for file-based configuration | Application must know the file path and how to reload it. |

# 5. Application Architecture




ConfigMap supplies normal settings; Secret supplies sensitive values.
The application consumes both through Kubernetes-managed configuration.

The application container will expose an HTML page generated at startup. The page shows a non-sensitive application mode and a masked indication that a sensitive secret was successfully injected. The actual secret value should not be displayed.

# 6. Kubernetes Objects Used

| Object | Name | Purpose |
|---|---|---|
| ConfigMap | app-config | Stores APP_MODE and WELCOME_MESSAGE. |
| Secret | app-secret | Stores DB_USERNAME and DB_PASSWORD. |
| Deployment | config-app | Runs the containerized application. |
| Service (optional) | config-app-service | Provides a stable local access endpoint if using NodePort. |

# 7. Procedure

## Step 1 - Verify the Cluster

1. Open PowerShell, Command Prompt, or terminal.
2. Verify the kubectl client and active cluster.

```
kubectl version --client
kubectl config current-context
kubectl get nodes
```

**Expected result:** The cluster is reachable and at least one node is Ready.

## Step 2 - Create a Working Namespace

```
kubectl create namespace config-lab
kubectl config set-context --current --namespace=config-lab
kubectl get all
```

All objects in the experiment are now created in the config-lab namespace.

## Step 3 - Create the ConfigMap

Create a file named app-config.yaml:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_MODE: "development"
  WELCOME_MESSAGE: "Hello from Kubernetes ConfigMap"
```

Apply it:

```
kubectl apply -f app-config.yaml
kubectl get configmap app-config
kubectl describe configmap app-config
```

## Step 4 - Create the Secret

Create a local file named app-secret.yaml. Use only dummy values for this academic experiment:

```
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
stringData:
  DB_USERNAME: "labuser"
  DB_PASSWORD: "LabPass@123"
```

**Security note:** stringData lets you write plaintext values in the manifest and Kubernetes converts them into the Secret data representation. Do not commit this file to a public source repository when it contains real credentials.

Apply and inspect without exposing the plaintext value:

```
kubectl apply -f app-secret.yaml
kubectl get secret app-secret
kubectl describe secret app-secret
```

## Step 5 - Build the Application Deployment

Create a file named config-app-deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: config-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: config-app
  template:
    metadata:
      labels:
        app: config-app
    spec:
      containers:
        - name: web
          image: nginx:alpine
          command: ["/bin/sh", "-c"]
          args:
            - |
              cat > /usr/share/nginx/html/index.html <<EOF
              <html>
              <body>
```

```
                <h1>Configuration Management Demo</h1>
                <p>Mode: ${APP_MODE}</p>
                <p>${WELCOME_MESSAGE}</p>
                <p>DB User configured: ${DB_USERNAME}</p>
                <p>DB Password: [MASKED]</p>
              </body>
              </html>
              EOF
              nginx -g 'daemon off;'
          env:
            - name: APP_MODE
              valueFrom:
                configMapKeyRef:
                  name: app-config
                  key: APP_MODE
            - name: WELCOME_MESSAGE
              valueFrom:
                configMapKeyRef:
                  name: app-config
                  key: WELCOME_MESSAGE
            - name: DB_USERNAME
              valueFrom:
                secretKeyRef:
                  name: app-secret
                  key: DB_USERNAME
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: app-secret
                  key: DB_PASSWORD
          ports:
            - containerPort: 80
```

Apply the Deployment:

```
kubectl apply -f config-app-deployment.yaml
kubectl get deployment
kubectl get pods -l app=config-app
```

## Step 6 - Verify That the Container Received the Configuration

Select one running Pod:

```
kubectl get pods -l app=config-app
```

Then check the injected environment variables. Do not print the password in a submitted report.

```
kubectl exec <POD_NAME> -- printenv APP_MODE
kubectl exec <POD_NAME> -- printenv WELCOME_MESSAGE
kubectl exec <POD_NAME> -- printenv DB_USERNAME
```

**Do not run in the submitted screenshot:** kubectl exec <POD_NAME> -- printenv DB_PASSWORD. The lab is intended to teach safe handling, so students should avoid revealing the secret value in screenshots or reports.

## Step 7 - Expose the Application for Testing

Create app-service.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: config-app-service
spec:
  type: NodePort
  selector:
    app: config-app
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

```
kubectl apply -f app-service.yaml
kubectl get service config-app-service
```

On Docker Desktop, you can test the service using http://localhost:30080 when the NodePort is reachable in your local setup. If NodePort access differs in your environment, use port-forwarding instead:

```
kubectl port-forward service/config-app-service 8080:80
```

Open http://localhost:8080 in a browser.

**Expected result:** The page should show the non-sensitive values and the DB username. The password should remain masked.

## Step 8 - Demonstrate a Secret Mounted as a File

Add the following volume and volumeMount definitions to the container specification to demonstrate the file-based method:

```
volumeMounts:
  - name: secret-volume
    mountPath: /etc/app-secret
    readOnly: true

volumes:
  - name: secret-volume
    secret:
      secretName: app-secret
```

After updating the Deployment, verify the files:

```
kubectl exec <POD_NAME> -- ls -l /etc/app-secret
kubectl exec <POD_NAME> -- sh -c 'cat /etc/app-secret/DB_USERNAME'
```

**Security practice:** Avoid printing /etc/app-secret/DB_PASSWORD during the lab. The point is to prove the file exists and can be consumed, not to expose its content.

## Step 9 - Demonstrate a ConfigMap Mounted as a File

Add the following definitions to the same Deployment to mount configuration values as files:

```
volumeMounts:
  - name: config-volume
    mountPath: /etc/app-config
    readOnly: true

volumes:
  - name: config-volume
    configMap:
      name: app-config
```

Verify:

```
kubectl exec <POD_NAME> -- ls -l /etc/app-config
kubectl exec <POD_NAME> -- cat /etc/app-config/APP_MODE
```

Students should compare environment-variable consumption with file-mounted consumption and explain which method is more convenient for a given application.

## Step 10 - Update Non-Sensitive Configuration

Edit the ConfigMap:

```
kubectl edit configmap app-config
```

Change APP_MODE from "development" to "testing". Then inspect:

```
kubectl get configmap app-config -o yaml
```

**Important observation:** Existing containers that consume a ConfigMap through environment variables do not automatically receive the new value; the Pod generally needs to be recreated. Mounted ConfigMap files behave differently and are updated by Kubernetes after a propagation delay, although applications may still need to reload the file to use the new value.

### Step 11 - Restart the Deployment to Consume Updated Environment Variables

```
kubectl rollout restart deployment config-app
kubectl rollout status deployment config-app
kubectl get pods -l app=config-app
```

Verify the new non-sensitive value:

```
kubectl exec <POD_NAME> -- printenv APP_MODE
```

### Step 12 - Inspect the Secret Safely

```
kubectl get secret app-secret
kubectl get secret app-secret -o yaml
```

The output uses encoded data. Demonstrate one value only if instructed by the faculty; otherwise avoid decoding credentials. For conceptual understanding, remember that base64 is not encryption.

```
kubectl get secret app-secret -o jsonpath='{.data.DB_USERNAME}'
```

**Do not decode real credentials:** Never paste real production credentials into a lab report, chat, issue tracker, or source repository.

### Step 13 - Troubleshooting Exercise

Temporarily change the ConfigMap key APP_MODE to another name such as APPLICATION_MODE while leaving the Deployment reference as APP_MODE.

```
kubectl edit configmap app-config
```

Restart the Deployment and inspect Pod status:

```
kubectl rollout restart deployment config-app
kubectl get pods
kubectl describe pod <POD_NAME>
```

Identify why the application cannot obtain the expected ConfigMap key. Restore APP_MODE and restart the Deployment.

**Learning point:** Configuration references are contracts: the key name in the consuming Deployment must match the key exposed by the ConfigMap or Secret.

### Step 14 - Cleanup

```
kubectl delete service config-app-service
kubectl delete deployment config-app
kubectl delete configmap app-config
kubectl delete secret app-secret
kubectl config set-context --current --namespace=default
kubectl delete namespace config-lab
```

## 8. Observation Sheet

| Activity | What to record | Expected observation |
|---|---|---|
| Cluster verification | Context + node status | Cluster is reachable. |
| ConfigMap | Keys APP_MODE and WELCOME_MESSAGE | Two non-sensitive settings are stored. |
| Secret | Secret keys | Sensitive data stored separately. |
| Deployment | Replica count + Pod names | Two application Pods become Ready. |
| Environment injection | APP_MODE + DB_USERNAME | Container receives values from both objects. |

| | | |
|---|---|---|
| Web test | Browser/port-forward result | App displays ConfigMap data and masked secret indicator. |
| Mounted files | File names under /etc/app-config and /etc/app-secret | Both objects are available as files. |
| ConfigMap update | Old vs new APP_MODE | Environment-based value changes after Pod restart. |
| Troubleshooting | Error caused by wrong key | Student identifies the missing configuration key. |

## 9. YAML Field Explanation

| Field | Meaning |
|---|---|
| ConfigMap.data | Stores non-sensitive key-value pairs. |
| Secret.stringData | Accepts plaintext input that Kubernetes converts for Secret storage. |
| secretKeyRef | Injects a specific Secret key as an environment variable. |
| configMapKeyRef | Injects a specific ConfigMap key as an environment variable. |
| configMap volume | Makes ConfigMap entries available as files. |
| secret volume | Makes Secret entries available as files. |
| readOnly: true | Prevents the container from modifying the mounted files. |
| rollout restart | Recreates Pods so environment variables are populated from updated values. |

## 10. Troubleshooting Guide

| Problem | Useful command | Likely cause / action |
|---|---|---|
| Pod not Ready | kubectl describe pod <pod> | Check image pull, container command, resources, and events. |
| ConfigMap key error | kubectl describe pod <pod> | Referenced key is missing or object name is incorrect. |
| Secret key error | kubectl describe pod <pod> | Referenced Secret key/name is incorrect. |
| Page shows blank value | kubectl exec <pod> -- printenv <NAME> | Environment variable not injected or application template is wrong. |
| Service unavailable | kubectl get svc; kubectl get endpoints | Check selector, Pods, and targetPort. |
| Port-forward fails | kubectl get pods; kubectl get svc | Check that the Pod is Ready and the Service exists. |
| Mounted file missing | kubectl exec <pod> -- ls -l <path> | Check volume and volumeMount names. |

## 11. Security and Good Practices

- Do not place passwords, API keys, or cloud credentials directly in container images.
- Do not store sensitive values in a ConfigMap when a Secret is appropriate.
- Do not assume base64 encoding makes a Secret confidential; protect access to the Kubernetes API and Secret objects.
- Use least-privilege RBAC so workloads and users can access only the configuration they need.
- Keep real credentials outside public Git repositories and student submissions.
- Use an external secret-management solution or stronger envelope/encryption controls when required by the production environment.
- Avoid printing secrets to logs, terminal history, screenshots, or lab reports.

## 14. Student Submission Format

- Title, objective, and brief theory.
- app-config.yaml and the relevant Deployment YAML.
- Secret definition using dummy credentials only.
- Screenshots of ConfigMap and Secret creation.

- Screenshot proving both Pods are Ready.
- Screenshot of environment variables showing non-sensitive values and DB username only.
- Screenshot of the application page with the password masked.
- Evidence of mounted ConfigMap and Secret files.
- Troubleshooting observation and corrective action.
- Answers to analysis and viva questions.
- Conclusion describing how configuration management improves portability and maintainability.

## 15. Assessment Rubric

| Criterion | Marks | Performance indicator |
|---|---|---|
| ConfigMap + Secret creation | 15 | Correct objects, keys, and separation of sensitive/non-sensitive data. |
| Deployment configuration | 15 | Correct references using configMapKeyRef and secretKeyRef. |
| Application validation | 15 | Application successfully consumes both configuration types. |
| Mounted-file demonstration | 10 | Correct volume and volumeMount configuration. |
| Update + restart test | 10 | Explains behavior after ConfigMap change. |
| Troubleshooting | 10 | Identifies and fixes configuration-key mismatch. |
| Security practices | 10 | Uses dummy values and avoids exposing secret data. |
| Documentation | 5 | Clear screenshots, observations, and conclusion. |
| Viva | 10 | Correct conceptual understanding. |

## 16. Extension Activities

- Create a second ConfigMap for a production-like profile and switch the Deployment between profiles.
- Use a single Secret with multiple keys, such as DB_USERNAME, DB_PASSWORD, and API_TOKEN.
- Convert the Deployment to consume all configuration through mounted files instead of environment variables.
- Add a readiness probe and study how configuration errors affect Pod readiness.
- Integrate a small application such as Node.js or Python that reads APP_MODE and DB_PASSWORD from the environment.
- Compare Kubernetes Secret management with an external secret manager as a discussion exercise.

## 17. Expected Learning Outcome

After completing the experiment, students should be able to design a Kubernetes Deployment that separates application code from environment-specific configuration, use ConfigMaps for non-sensitive values, use Secrets for sensitive values, and validate both environment-based and file-based configuration consumption while following basic security practices.

## 18. References


- Kubernetes Documentation - ConfigMaps: https://kubernetes.io/docs/concepts/configuration/configmap/
- Kubernetes Documentation - Secrets: https://kubernetes.io/docs/concepts/configuration/secret/
- Kubernetes Documentation - Configure a Pod to Use a ConfigMap: https://kubernetes.io/docs/tasks/configure-pod-container/configure-pod-configmap/
- Kubernetes Documentation - Configure a Pod to Use a Secret: https://kubernetes.io/docs/tasks/inject-data-application/distribute-credentials-secure/